

# SVG Icon Agent: A Collaborative LLM-Agent Pipeline for Editable SVG Icon Generation

Guanjie Zhao

Student ID: 23307130127

Fudan University

[https://github.com/fwarlnben/CG\\_PJ](https://github.com/fwarlnben/CG_PJ)

## Abstract

This report presents SVG Icon Agent, a course project for generative AI practice in computer graphics. The system converts short English icon prompts into editable SVG assets through a collaborative LLM-agent pipeline rather than a raster text-to-image model. It decomposes icon creation into goal management, local memory retrieval, prompt rewriting, structured planning, multi-candidate SVG drafting, semantic and SVG-quality critique, consensus selection, optimization, validation, failure taxonomy, repair routing, refinement, memory curation, and gallery export. The graphics output remains inspectable as SVG source code, while deterministic local tools check parseability, safety, canvas constraints, primitive usage, rendering risk, and export artifacts. In the current Web UI experiment snapshot, 15 single-prompt runs generated 45 candidate SVGs and 15 selected winners. The optimized baseline icons were valid in 13/15 cases, while the final refined icons were valid in 15/15 cases; the average validation score improved from 89.33 to 93.20. Code is available at [https://github.com/fwarlnben/CG\\_PJ](https://github.com/fwarlnben/CG_PJ).

## 1 Introduction

Generative AI systems in graphics are often demonstrated through impressive raster images, 3D scenes, or videos. However, many daily graphics tasks are smaller and more structural: a user may need a clean toolbar icon, a notification badge, a security symbol, or a simple object pictogram that can be edited after generation. For these cases, the output format matters. A PNG image can look attractive, but it is difficult to inspect, repair, recolor, or integrate into a design system. An SVG icon, in contrast, exposes its geometry, colors, accessibility metadata, and canvas constraints directly as editable vector code [6].

SVG Icon Agent explores this narrower but practical setting. The project asks whether a language-model-driven agent workflow can generate usable editable SVG icons by decomposing the task into explicit subproblems. Instead of asking one model call to “draw an icon”, the system separates goal formation, prompt improvement, planning, candidate generation, independent critique, consensus selection, validation, and targeted repair. This design follows a central observation from recent graphics agents: generation quality improves when high-level intent, intermediate structure, visual criticism, and repair strategy are represented explicitly [3, 4, 7].

The main contributions of this project are:

- an end-to-end multi-agent workflow for editable text-to-SVG icon generation;
- a retrieval-augmented goal-conditioning step that reuses local historical run memories;
- a candidate–critic–selector stage that makes selection inspectable instead of relying on a single sample;
- a validation and repair loop with failure taxonomy and route selection before refinement;
- a Web UI that exposes agent progress, generated artifacts, raw traces, post-run optimizer feedback, and an SVG source editor.

## 2 Related Work

**Prompt optimization and 2D generation agents.** GenPilot decomposes text-to-image prompt optimization into error detection, prompt improvement, multi-candidate generation, scoring, and memory [7]. PromptSculptor similarly studies multi-agent prompt optimization for image generation [5]. These systems motivated the idea that visual generation should not be treated as a single opaque prompt-to-output call. SVG Icon Agent adapts this philosophy to SVG icon generation, where the output can be checked as source code and repaired structurally.

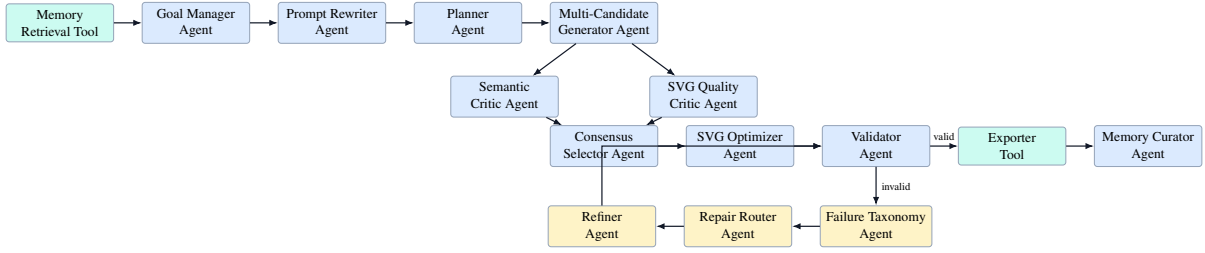


Figure 1: The SVG Icon Agent workflow. Blue nodes are LLM-backed agents, green nodes are deterministic tools, and yellow nodes form the conditional repair route.

**Layout and poster agents.** Paper2Poster turns long scientific papers into posters through parsing, planning, layout, and visual feedback [3]. PosterForest further emphasizes hierarchical multi-agent collaboration for poster generation [4]. Although this project targets icon-sized graphics rather than large posters, it shares the same principle: split graphics generation into content understanding, structured planning, rendering, critique, and revision.

**Tool-augmented graphics agents.** Agentic 3D and CAD systems such as CAD-Assistant show that language models become more useful for graphics tasks when they can interact with domain-specific tools and constraints [1]. SVG Icon Agent follows this tool-augmented pattern in a lightweight 2D vector domain. The LLM agents make semantic and design decisions, while local tools perform deterministic SVG checks, render previews, and export reports.

## 3 Method

### 3.1 Task Definition

The input is a short English prompt describing an icon, such as a rocket launch icon, a steaming coffee cup, or a security shield. The desired output is a complete SVG file that is:

- valid XML/SVG with a square canvas and explicit `viewBox`;
- editable through standard SVG primitives and paths;
- safe, with no scripts, external references, iframes, animation tags, or foreign objects;
- visually recognizable at small icon sizes;
- exported with PNG previews, validation reports, traces, and gallery pages.

### 3.2 Agent Pipeline

Figure 1 shows the core workflow. Named agents call the configured LLM backend through OpenRouter [2]; deterministic local modules are treated as tools rather than agents. The default model configured in the project is `openai/gpt-oss-120b:free`.

### 3.3 Retrieval and Goal Conditioning

The MEMORY RETRIEVAL TOOL searches local memory records from previous runs. Each memory record contains a prompt summary, successful strategies, failure patterns, user feedback, tags, and scores. The GOAL MANAGER AGENT then converts the raw user prompt, optional user goal, and retrieved memory context into explicit objectives, visual requirements, constraints, acceptance criteria, preferences, and avoid patterns. This step makes the generation target observable before any SVG is drafted.

### 3.4 Planning, Candidate Generation, and Critique

The PROMPT REWRITER AGENT rewrites the user input into a concise SVG-icon prompt. The PLANNER AGENT extracts an icon category, motifs, palette, layout, and constraints. The MULTI-CANDIDATE GENERATOR AGENT then produces three distinct SVG drafts for each prompt in collaborative mode. Two independent critics evaluate the candidates:

- the SEMANTIC CRITIC AGENT scores prompt alignment, recognizability, and small-size readability;
- the SVG QUALITY CRITIC AGENT scores editability, safety, rendering risk, and deterministic SVG check issues.

The CONSENSUS SELECTOR AGENT selects the strongest draft and writes a repair brief for downstream optimization.

### 3.5 Optimization, Validation, and Repair

The selected SVG is not immediately accepted. The SVG OPTIMIZER AGENT rewrites it using critic reports, selector risks, deterministic tool findings, and optional human feedback. The VALIDATOR AGENT then judges semantic alignment, visual quality, editability, and rule compliance. If validation finds blocking issues, the pipeline invokes:

1. the FAILURE TAXONOMY AGENT, which classifies failure types and root causes;
2. the REPAIR ROUTER AGENT, which chooses an ordered repair strategy such as safety rebuild, semantic recomposition, simplification, layout rebalance, or minor patch;
3. the REFINER AGENT, which returns a complete repaired SVG.

The repaired SVG is sent back to validation until accepted or the configured repair-round limit is reached.

## 4 Implementation

The implementation is a Python project organized around the `svg_icon_agent` package. The CLI entry point supports JSON prompt cases, selected cases, manual text prompts, and interactive prompts. The Web UI writes timestamped runs under `outputs/web/`, exposes live agent states, displays candidate/selected/baseline/refined SVGs, and provides a source-level SVG editor that saves separate edited artifacts without overwriting generated outputs.

Local code intentionally does not synthesize fallback icons when a model call fails. Instead, it records errors in trace files so that failures remain inspectable. The deterministic toolchain performs prompt loading, memory retrieval, SVG safety and structure checks, PNG rendering, metric aggregation, HTML gallery export, and raw response logging. Generated outputs include candidate SVGs, selected SVGs, optimized baselines, refined SVGs, PNG previews, validation JSON, failure taxonomies, repair routes, memory context, generation goals, LLM traces, and raw LLM response logs.

## 5 Experiments and Results

### 5.1 Setup

The current evaluation uses the completed Web UI runs under `outputs/web/`. Each run is a single-prompt icon generation case using collaborative mode with three candidates. The snapshot includes 15 runs, 45 candidate SVG drafts, and 15 selected winners. This is a small project evaluation intended to verify the full workflow, not a large-scale benchmark.

The main metrics are baseline validity, final refined validity, validation score, score delta, and maximum repair rounds. The baseline is the optimized SVG after candidate selection and optimizer feedback. The refined result is the output after conditional validation and repair.

### 5.2 Repair Examples

Table 2 lists representative repair cases from the run logs. The coffee and dog cases show the clearest effect of the validation–taxonomy–routing–refinement loop. The apple case is included as an honest caveat: a refined result can be accepted while receiving a lower validation score, which suggests that future work should combine automatic validation with human preference evaluation.

Table 1: Current Web UI experiment snapshot.

| Metric                            | Value |
|-----------------------------------|-------|
| Web UI single-prompt runs         | 15    |
| Generated candidate SVGs          | 45    |
| Selected winners                  | 15    |
| Valid optimized baselines         | 13/15 |
| Valid final refined icons         | 15/15 |
| Baseline average validation score | 89.33 |
| Refined average validation score  | 93.20 |
| Average score delta               | +3.87 |
| Maximum repair rounds used        | 3     |

Table 2: Representative validation-score changes after repair.

| Case                | Baseline | Refined | Rounds | Observation                                  |
|---------------------|----------|---------|--------|----------------------------------------------|
| Steaming coffee cup | 70       | 95      | 1      | Large semantic and visual repair gain        |
| Cute yellow dog     | 65       | 100     | 1      | Strong improvement after targeted refinement |
| Clean line icon     | 88       | 95      | 3      | Multi-round repair improves structure        |
| Apple icon          | 85       | 70      | 1      | Accepted but lower score; metric caveat      |

### 5.3 Generated Examples

Figure 2 shows generated refined icons selected from the project outputs. The examples cover object icons, UI symbols, education/productivity symbols, and simple scene-like icons. They demonstrate that the system can create compact vector assets with clean silhouettes and small-size readability.

## 6 Discussion and Limitations

The experiment suggests that an agentic SVG workflow can improve validity and inspectability compared with accepting the first generated draft. The explicit repair loop is especially useful when the baseline icon is semantically close but structurally flawed, visually cluttered, or unsafe. The Web UI also makes the system easier to demonstrate because users can inspect rewritten prompts, retrieved memory, candidates, selected winners, validation issues, repair routes, raw responses, and final SVG source.

Several limitations remain. First, the current evaluation is small: 15 single-prompt Web runs are enough to validate the workflow but not enough for a robust benchmark. Second, the validation score is LLM-assisted and should be complemented by human preference studies. Third, the local `SVGCHECKTOOL` covers a practical safe subset of SVG rather than the entire SVG standard. Fourth, the system depends on the availability and quality of the configured LLM service; free API models can be slow, rate-limited, or inconsistent. Finally, the current generator focuses on simple icon graphics rather than complex illustration, typography, or full layout design.

## 7 Conclusion

SVG Icon Agent demonstrates a practical agent-based approach to editable vector graphics generation. By decomposing text-to-SVG icon creation into goal conditioning, prompt rewriting, planning, candidate generation, critique, selection, optimization, validation, taxonomy, routing, and refinement, the system makes generation more inspectable and repairable. The current project snapshot improves valid icons from 13/15 optimized baselines to 15/15 final refined results, with an average validation-score increase from 89.33 to 93.20. Future work should expand the prompt set, add human preference evaluation, compare more ablation settings such as single-shot generation and no-memory generation, and broaden the SVG grammar while preserving safety and editability.

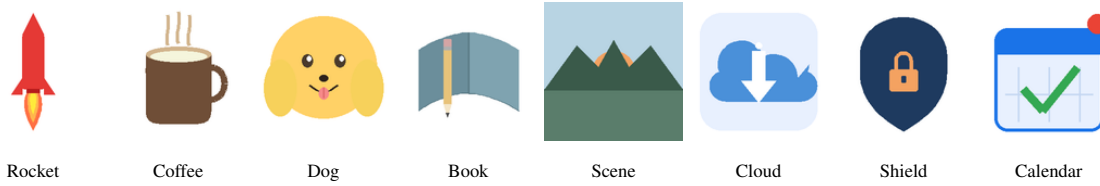


Figure 2: Example refined icon outputs used for visual inspection in the project report and presentation materials.

## Member Contributions

**Guanjie Zhao** (Student ID: **23307130127**) completed this as a solo project. The work included topic selection, system design, implementation of the multi-agent SVG pipeline, Web UI development, SVG editor integration, experiment execution, evaluation, artifact organization, report writing, and presentation preparation.

## References

- [1] Dimitrios Mallis et al. Cad-assistant: Tool-augmented vlms as generic cad task solvers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. URL <https://cadassistant.github.io/>.
- [2] OpenRouter. Openrouter api documentation, 2026. URL <https://openrouter.ai/docs/>.
- [3] Paper2Poster Team. Paper2poster: Towards multimodal poster automation from scientific papers. *arXiv preprint arXiv:2505.21497*, 2025. URL <https://arxiv.org/abs/2505.21497>.
- [4] PosterForest Team. Posterforest: Hierarchical multi-agent collaboration for scientific poster generation. *arXiv preprint arXiv:2508.21720*, 2025. URL <https://arxiv.org/abs/2508.21720>.
- [5] Yixuan Wang et al. Promptsculptor: Multi-agent based text-to-image prompt optimization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2025. URL <https://aclanthology.org/2025.emnlp-demos.59/>.
- [6] World Wide Web Consortium. Scalable vector graphics (svg) 1.1 specification. *W3C Recommendation*, 2011. URL <https://www.w3.org/TR/SVG11/>.
- [7] Yuwei Yang et al. Genpilot: A multi-agent system for test-time prompt optimization in image generation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2025. URL <https://aclanthology.org/2025.findings-emnlp.49/>.